94.3
Pass

Sairam Badiga

PDF generated at: 13 Feb 2026 06:01:01 UTC

View this report on HackerRank

Score

94.3% • 165.0 / 175
scored in eClerx - Java Coding Test - 3+ Years New in 19 min 48 sec on 13 Feb 2026 11:05:32 IST

Candidate Information

Email	sairambadiga1995@gmail.com
Test	eClerx - Java Coding Test - 3+ Years New
Candidate Packet	View
Taken on	13 Feb 2026 11:05:32 IST
Time taken	19 min 48 sec/ 90 min
Work Experience	> 5 years
Invited by	Nancy
Invited on	12 Feb 2026 10:21:09 IST

Skill Distribution

No.	Skill	Score
1	Spring Boot Advanced	0%

2	Spring Boot Intermediate	75%
3	Problem Solving Intermediate	100%
4	Java Intermediate	100%

Tags Distribution

Hard	0%	Spring Boot	60%
TDD	0%	Spring	0%
Application Properties Annotations	0%	Medium	97%
Pointers	100%	Java	100%
Exceptions	100%	OOP	100%

Questions

Spring Boot (Advanced) • 15 / 25

No.	Question	Skill	Score	Code Quality	Time Taken	Status
1	Output Multiple Choice	Spring Boot (Advanced)	0/5	-	1 min 21 sec	⊗

2	Configuration Properties Multiple Choice	Spring Boot (Intermediate)	0/5	-	44 sec	✗
3	Security Custom Login Multiple Choice	Spring Boot (Intermediate)	5/5	-	55 sec	✓
4	Endpoint Security Multiple Choice	Spring Boot (Intermediate)	5/5	-	29 sec	✓
5	CSRF Multiple Choice	Spring Boot (Intermediate)	5/5	-	27 sec	✓

Problem Solving (Intermediate) • 75 / 75

No.	Question	Skill	Score	Code Quality	Time Taken	Status
6	Quiz Competition Coding	Problem Solving (Intermediate)	75/75	-	5 min 21 sec	✓

Java (Intermediate) • 75 / 75

No.	Question	Skill	Score	Code Quality	Time Taken	Status
7	Java: Vending Machine Coding	Java (Intermediate)	75/75	-	10 min 10 sec	✓

1. Output

Incorrect

Multiple Choice | Hard | Spring Boot | TDD

Question description

What will happen to the output file?

```
@SpringBootTest
public class LogTest
{
   File log;
   FileWriter logWriter;

   @Autowired
   SomeClassToTest someClassToTest;

   @Before
   public void init()
   {
     log = new File("log.txt");
     logWriter = new FileWriter(log);
   }

   @After
   public void teardown()
   {
      log.delete();
   }

   @Test
   public void test1()
   {
      logWriter.write("Entered Test 1");
      assertTrue(someClassToTest.someMethod());
   }

   @Test
   public void test2()
   {
      logWriter.write("Entered Test 2");
      assertTrue(someClassToTest.otherMethod());
```

```
    }
}
```

Candidate's Solution

Options: (Expected answer indicated with a tick)

- ○ The output file is not created.
- ● **The output file is created but not deleted.**
- ○ The output file is created and eventually deleted. ✓
- ○ The create and delete blocks will not be executed.

No comments.

2. Configuration Properties

Incorrect

Multiple Choice | Spring | Spring Boot | Application Properties Annotations | Medium

Question description

An *application.properties* file contains the following properties:

```
app.prop1 = value1
```

```
app.prop2 = value2
```

You decided to create a class to hold this configuration. To protect it, the class is defined as final.

```
@Configuration
@ConfigurationProperties(prefix = "app")
public final class AppConfiguration {
  String prop1;
  String prop2;

  public String getProp1() {
    return prop1;
  }

  public void setProp1(String prop1) {
    this.prop1 = prop1;
  }

  public String getProp2() {
    return prop2;
  }

  public void setProp2(String prop2) {
    this.prop2 = prop2;
  }
}
```

What is the result of running the Spring application with this class?

Interviewer guidelines

Spring creates dynamic proxies for classes annotated with `@Configuration` classes. Spring uses CGLIB to extend your class to create proxy. Hence, configuration classes cannot be final. When running the code Spring throws `BeanDefinitionParsingException` and this is the correct option.

Candidate's Solution

Options: (Expected answer indicated with a tick)

- [x] The application runs, the configuration bean is created. and the properties are initialized correctly (prop1=value1, prop2=value2).
- [ ] The application runs, and a configuration bean is created, but properties are set to null (prop1=null, prop2=null).
- [ ] A BeanDefinitionParsingException is thrown because the @Configuration bean cannot be marked as final.
- [ ] An IllegalStateException is thrown because the @Configuration bean cannot have an @ConfigurationProperties annotation.
- [ ] A NoSuchMethodException is thrown because the @Configuration bean needs a default constructor.

No comments.

3. Security Custom Login

Correct

Multiple Choice | Spring Boot | Medium

Question description

Which of the following options can be used to create a custom login page in Spring Security?

Candidate's Solution

Options: (Expected answer indicated with a tick)

- protected void configure(HttpSecurity http) throws Exception { http .authorizeRequests() .anyRequest().authenticated() .and() .formLogin() .and() .httpBasic(); }
- protected void configure(HttpSecurity http) throws Exception { http .authorizeRequests() .anyRequest().authenticated() .and() .formLogin() .loginPage("/login"); }
- **protected void configure(HttpSecurity http) throws Exception { http .authorizeRequests() .anyRequest().authenticated() .and() .formLogin() .loginPage("/login") .permitAll(); }**
- None of the above

No comments.

4. Endpoint Security

Correct

Multiple Choice | Spring Boot | Medium

Question description

In Spring Boot Security, the option to enable secure endpoints with @PreAuthorize annotation is:

Interviewer guidelines

@EnableGlobalMethodSecurity by default doesn't enable this feature. To enable this prePostEnabled=true value must be set

Candidate's Solution

Options: (Expected answer indicated with a tick)

- ○ Configuration class annotated with @EnableGlobalMethodSecurity
- ○ Configuration class annotated with @EnableGlobalMethodSecurity(securedEnabled = true)
- ● **Configuration class annotated with @EnableGlobalMethodSecurity(prePostEnabled = true)** ✓
- ○ None above

No comments.

---

5. CSRF

Correct

Multiple Choice | Spring Boot | Medium

Question description

Which Java Spring-based code segment correctly configures CSRF?

Interviewer guidelines

CSRF Protection makes sure that server is not bombarded with unwanted calls. Spring Security enables some features by default like CSRF and Form based authentication.

Candidate's Solution

Options: (Expected answer indicated with a tick)

- private void manage(HttpSecurity http) {   http.csrf().allowed() }
- private void manage(HttpSecurity http) {   http.csrf() }
- private void manage(HttpSecurity http) {   http.csrf().enable() }
- ** None of the above. CSRF is configured by default when Spring Security Dependency is added. **

No comments.

6. Quiz Competition

Correct

Coding Medium Pointers

Question description

A class has students with various talents, each represented by an integer from 1 to *talentsCount*. You need to form teams for a quiz competition, where each team must have at least one member with each talent.

Teams must be formed from consecutive students in the array. For each possible starting position, determine the minimum number of students needed to form a valid team. If it is not possible to form a team with all talents from a particular starting position, return -1 for that position.

Example

talentsCount = 3

talent = [1, 2, 3, 2, 1]

- Starting at position 1: [1, 2, 3] includes all talents, minimum size = 3
- Starting at position 2: [2, 3, 2, 1] is the smallest subarray with all talents, minimum size = 4
- Starting at position 3: [3, 2, 1] includes all talents, minimum size = 3
- Starting at positions 4 and 5: Cannot form a team with all talents, return -1

The result is [3, 4, 3, -1, -1].

Function Description

Complete the function *teamSize* in the editor with the following parameter(s):

int talent[n]: the students' talents

int talentsCount : the number of talents represented numbered from 1 to *talentsCount*

Returns

int[n]: at each index, the minimum size subarray required, or -1 if an appropriate subarray does not exist

Constraints

- $1 \le n, talentsCount \le 10^5$
- $1 \le talent[i] \le talentsCount$

▼ INPUT FORMAT FOR CUSTOM TESTING

The first line contains an integer, *n*, the number of elements in *talent*.

Each line *i* of the *n* subsequent lines (where $0 \le i < n$) contains an integer, *talent[i]*.

The next line contains the integer *talentCount*.

▼ SAMPLE CASE 0

Sample Input For Custom Testing

```
STDIN   Function
-----   --------
8       talent[] size n = 8
1       talent = [1, 1, 2, 2, 3, 1, 3, 2]
1
2
2
3
1
3
2
3       talentsCount = 3
```

Sample Output

```
5
4
4
3
4
3
-1
-1
```

Explanation

The shortest subarrays for each position:

Beginning at the first position: [1, 1, 2, 2, 3] (length 5)

Second position: [1, 2, 2, 3] (length 4)

Third position: [2, 2, 3, 1] (length 4)

Fourth position: [2, 3, 1] (length 3)

Fifth position: [3, 1, 3, 2] (length 4)

Sixth position: [1, 3, 2] (length 3)

No further subarrays will have 3 elements.

▼ SAMPLE CASE 1

Sample Input For Custom Testing

```
STDIN   Function
-----   --------
5       talent[] size n = 5
1       talent = [1, 1, 1, 1, 1]
1
```

```
1
1
1
1      talentsCount = 1
```

Sample Output

```
1
1
1
1
1
```

Explanation

Since there is only 1 type of talent, any 1 person can form a team.

▼ SAMPLE CASE 2

Sample Input For Custom Testing

```
STDIN   Function
-----   --------
9       talent[] size n = 9
7       talent = [7, 5, 3, 4, 6, 1, 7, 2, 4]
5
3
4
6
1
7
2
4
7       talentsCount  7
```

Sample Output

```
8
7
-1
-1
-1
-1
-1
-1
-1
```

Explanation

There have to be at least *7* elements in any subarray to have one from each talent. The shortest subarrays at each position are:
Beginning at the first position: [7, 5, 3, 4, 6, 1, 7, 2] (length 8)
Second position: [5, 3, 4, 6, 1, 7, 2] (length 7)

No further subarrays will have 7 elements.

Interviewer guidelines

▼ SOLUTION

This can be solved using two-pointer technique. For a particular left index (L), we move the right index (R) forward until we have k distinct elements, then once we get it, we can move L forward. Each element will be traversed at most two times.

Time Complexity: O(n) or O(nLogn) (with set/map).

C++ Solution:

```
int f[100005];

vector<int> teamSize(vector<int> talent, int talentsCount) {
    int k = talentsCount;
    int n = talent.size();
    assert(n>=1&&n<=100000);
    assert(k>=1&&k<=n);
    for(int i=0;i<n;i++)
    {
        assert(talent[i]<=k&&talent[i]>=1);
    }
    int cnt = 0;
    int j = 0;
    vector<int> ans(n);
    for(int i=0;i<n;i++)
    {
        int x = talent[i];
        f[x]++;
        if(f[x]==1)
            cnt++;
        while(cnt==k&&j<=i)
```

```
        {
            ans[j] = i - j + 1;
            int y = talent[j];
            f[y]--;
            if(f[y]==0)
                cnt--;
            j++;
        }
    }
    while(j<n)
        ans[j++] = -1;
    return ans;
}
```

Candidate's Solution

Language used: **Java 21**

```java
import java.io.*;
import java.math.*;
import java.security.*;
import java.text.*;
import java.util.*;
import java.util.concurrent.*;
import java.util.function.*;
import java.util.regex.*;
import java.util.stream.*;
import static java.util.stream.Collectors.joining;
import static java.util.stream.Collectors.toList;


class Result {

    /*
     * Complete the 'teamSize' function below.
     *
     * The function is expected to return an INTEGER_ARRAY.
     * The function accepts following parameters:
     *  1. INTEGER_ARRAY talent
     *  2. INTEGER talentsCount
     */

```

```java
    public static List<Integer> teamSize(List<Integer> talent, int talentsCount) {
    // Write your code here
    int n = talent.size();
    int[] freq = new int[talentsCount + 1];
    int have = 0;
    int right = 0;
    List<Integer> result = new ArrayList<>();
    for(int left = 0; left<n; left++){
        while(right < n && have < talentsCount){
            int t = talent.get(right);
            if(freq[t] == 0) have++;
            freq[t]++;
            right++;

        }
        if(have == talentsCount){
            result.add(right -left);

        }
        else{
            result.add(-1);
        }
        int t = talent.get(left);
        freq[t]--;
        if(freq[t] == 0) have--;
    }
    return result;

    }

}

public class Solution {
    public static void main(String[] args) throws IOException {
        BufferedReader bufferedReader = new BufferedReader(new InputStreamReader(System.in));
        BufferedWriter bufferedWriter = new BufferedWriter(new FileWriter(System.getenv("OUTPUT_PATH")));

        int talentCount = Integer.parseInt(bufferedReader.readLine().trim());

        List<Integer> talent = IntStream.range(0, talentCount).mapToObj(i -> {
            try {
                return bufferedReader.readLine().replaceAll("\\s+$", "");
```

```
            } catch (IOException ex) {
                throw new RuntimeException(ex);
            }
        })
            .map(String::trim)
            .map(Integer::parseInt)
            .collect(toList());

        int talentsCount = Integer.parseInt(bufferedReader.readLine().trim());

        List<Integer> result = Result.teamSize(talent, talentsCount);

        bufferedWriter.write(
            result.stream()
                .map(Object::toString)
                .collect(joining("\n"))
            + "\n"
        );

        bufferedReader.close();
        bufferedWriter.close();
    }
}

```

TESTCASE	DIFFICULTY	TYPE	STATUS	SCORE	TIME TAKEN	MEMORY USED
TestCase 0	Easy	Sample	Success	1	0.0594 sec	37.6 KB
TestCase 1	Easy	Sample	Success	1	0.055 sec	37.4 KB
TestCase 2	Easy	Sample	Success	1	0.0502 sec	37.8 KB
TestCase 3	Easy	Hidden	Success	3	0.1085 sec	49.2 KB
TestCase 4	Easy	Hidden	Success	3	0.1329 sec	49.5 KB

TestCase 5	Easy	Hidden	Success	3	0.1125 sec	49.3 KB
TestCase 6	Easy	Hidden	Success	8	0.4123 sec	71.7 KB
TestCase 7	Easy	Hidden	Success	8	0.3974 sec	70.2 KB
TestCase 8	Easy	Hidden	Success	8	0.3419 sec	71.1 KB
TestCase 9	Easy	Hidden	Success	8	0.3568 sec	70.7 KB
TestCase 10	Easy	Hidden	Success	8	0.3159 sec	70.1 KB
TestCase 11	Easy	Hidden	Success	8	0.2751 sec	71.6 KB
TestCase 12	Easy	Hidden	Success	8	0.3519 sec	69.8 KB
TestCase 13	Easy	Hidden	Success	7	0.3146 sec	69.7 KB

No comments.

7. Java: Vending Machine

Correct

Coding Medium Java Exceptions OOP

Question description

Design a vending machine that serves 3 drinks: "Coke", "Fanta", and "Sprite".

Create a *VendingMachine* class that implements the following methods:

- *void registerDrink(int buttonNumber, Drink drink)*: Registers a type of drink with a particular *buttonNumber* of the vending machine. Here, *Drink* is an interface provided in the locked stub.
- *ServedDrinkSummary(int buttonNumber, int money)*: Returns the Drink that is served when *buttonNumber* is pressed, and an amount of money is entered by a customer.

The class contains two attributes:

- *change*: the units of currency to return to the consumer
- *drink*: the drink requested

There are three potential error conditions:

- If the machine has run out of the drink, an *OutOfStockException* should be thrown with the message "{drinkName} is out of stock".
- If the money the user enters is insufficient to buy the drink, an *InsufficientMoneyException* should be thrown.
- If the drink is out of stock and the user has not entered enough money, throw an *OutOfStockException* with the previously mentioned message.

The *OutOfStockException, InsufficientMoneyException*, and *ServedDrinkSummary* classes are provided in the locked stub.

The locked stub validates the *VendingMachine* class by processing some requests.

Constraints

- $1 \le totalNumberOfRequests \le 10^4$
- $0 \le buttonNumber \le 2$
- $0 \le money \le 100$
- $0 \le quantity$ available of each $drink \le 10^4$
- $0 \le price$ per unit of each $drink \le 10^4$
- No two drinks are assigned the same *buttonNumber.*

▼ INPUT FORMAT FOR CUSTOM TESTING

The first line contains three integers that represent the *quantity* of *Coke*, *Fanta,* and *Sprite* in the vending machine respectively.

The second line contains three integers that represent the *price* per unit of *Coke*, *Fanta,* and *Sprite* respectively.

The third line contains three integers that represent the *buttonNumbers* assigned to *Coke*, *Fanta,* and *Sprite* respectively.

The fourth line contains an integer, *totalNumberOfRequests .*

The next *totalNumberOfRequests* lines contain the *buttonNumber* pressed and *money* inserted separated by a space.

▼ SAMPLE CASE 0

Sample Input For Custom Testing

```
STDIN    Function
-----    --------
2 1 0    quantity Coke = 2, Fanta = 1, Sprite = 0
5 10 15  price Coke = 5, Fanta = 10, Sprite = 15
0 1 2    buttonNumbers Coke = 0, Fanta = 1, Sprite = 2
4        totalNumberOfRequests = 4
0 20     buttonNumber = 0, money = 20 for the first request
2 20     buttonNumber = 2, money = 20 for the second request
2 5      ...
1 5
```

Sample Output

```
Vending machine set up
Coke 15
Sprite is out of stock
Sprite is out of stock
Insufficient money
```

Explanation

- *0 20:* A request to process a can of *Coke* and *20* units of money is provided to the vending machine. *Coke* is in stock and costs *5* units of money so *15* units in change is returned to the consumer. The locked stub prints "*Coke 15".*
- *2 20:* A request to process a can of *Sprite* and *20* units of money is provided to the vending machine. *Sprite* is out of stock so an *OutOfStockException* is thrown, and the stub prints "*Sprite is out of stock".*
- *2 5:* A request to process a can of *Sprite* and *5* units of money is provided to the vending machine. *Sprite* is out of stock and the amount of *money* is insufficient so an *OutOfStockException* is thrown and the stub prints, "*Sprite is out of stock".*

- *1 5:* A request to process a can of *Fanta* and *5* units of money is provided. Since the price of a unit of *Fanta is 10* and it is in stock, an *InsufficientMoneyException* is thrown, and the stub prints "*Insufficient money".*

Interviewer guidelines

▼ SOLUTION

Solution for Java 7, 8 and 15

```
class VendingMachine{
   HashMap<Integer, Drink> drinkRelatedToButton = new HashMap<>();

   void registerDrink(int buttonIdx, Drink drink){
      drinkRelatedToButton.put(buttonIdx, drink);
   }

   ServeDrinkSummary dispatch(int buttonPressed, int money)throws OutOfStockException, Insuffi
cientMoneyException{
      ServeDrinkSummary serveDrinkSummary = new ServeDrinkSummary();
      serveDrinkSummary.servedDrink = drinkRelatedToButton.get(buttonPressed);
      serveDrinkSummary.change = money - serveDrinkSummary.servedDrink.getPrice();

      if(serveDrinkSummary.servedDrink.getQuantityLeft() == 0){
         throw new OutOfStockException(serveDrinkSummary.servedDrink.getName() + " is out of st
ock");
      }

      if(serveDrinkSummary.change < 0){
         throw new InsufficientMoneyException();
      }
      return serveDrinkSummary;
   }
}
```

Candidate's Solution

Language used: **Java 21**

```
1 import java.io.*;
2 import java.util.*;
3 import java.text.*;
4 import java.math.*;
5 import java.util.regex.*;
6
7 interface Drink{
```

```
    int getPrice();
    String getName();
    int getQuantityLeft();
    void serveDrink();
}

class ServeDrinkSummary{
    Drink servedDrink;
    int change;
}

class OutOfStockException extends Exception{
    public OutOfStockException(String message){
        super(message);
    }
}

class InsufficientMoneyException extends Exception{
    public InsufficientMoneyException(){
        super();
    }
}
class VendingMachine{
    private static class Slot {
        Drink drink;
        int price;
        int quantity;
        Slot(Drink d){
            this.drink = d;
            this.price = d.getPrice();
            this.quantity = d.getQuantityLeft();

        }
    }
    private final Map<Integer, Slot> slots = new HashMap<>();
    void registerDrink(int buttonIdx, Drink drink){
        slots.put(buttonIdx, new Slot(drink));

    }


    ServeDrinkSummary dispatch(int buttonPressed, int money)
        throws OutOfStockException, InsufficientMoneyException{
            Slot slot = slots.get(buttonPressed);
            Drink drink = slot.drink;
            if(slot.quantity <= 0){
```

```
                throw new OutOfStockException(drink.getName() + " is out of stock");
            }
            if(money < slot.price){

                throw new InsufficientMoneyException();
            }
            slot.quantity--;
            int change = money -slot.price;
            ServeDrinkSummary summary = new ServeDrinkSummary();
            summary.servedDrink = drink;

            summary.change = change;
            return summary;
    }
}


class Solution{

    public static void main(String[] args) throws IOException{
        String arr[];
        int quantity[] = new int[3], pricePerUnit[] = new int[3], buttonAssigned[] = new int[3];

        BufferedReader br = new BufferedReader(new InputStreamReader(System.in));
        PrintWriter out = new PrintWriter(System.out);

        arr = br.readLine().split(" ");
        for(int i = 0; i < 3; i++){
            quantity[i] = Integer.parseInt(arr[i]);
        }

        arr = br.readLine().split(" ");
        for(int i = 0; i < 3; i++){
            pricePerUnit[i] = Integer.parseInt(arr[i]);
        }

        arr = br.readLine().split(" ");
        for(int i = 0; i < 3; i++){
            buttonAssigned[i] = Integer.parseInt(arr[i]);
        }

        VendingMachine vendingMachine = new VendingMachine();
        Drink coke = new Coke(pricePerUnit[0], "Coke", quantity[0]);
```

```
        Drink fanta = new Fanta(pricePerUnit[1], "Fanta", quantity[1]);
        Drink sprite = new Sprite(pricePerUnit[2], "Sprite", quantity[2]);
        vendingMachine.registerDrink(buttonAssigned[0], coke);
        vendingMachine.registerDrink(buttonAssigned[1], fanta);
        vendingMachine.registerDrink(buttonAssigned[2], sprite);
        out.println("Vending machine set up");

        int totalNumberOfRequests = Integer.parseInt(br.readLine().trim());
        while(totalNumberOfRequests-- > 0){
            arr = br.readLine().split(" ");
            int buttonPressed = Integer.parseInt(arr[0]),
                money = Integer.parseInt(arr[1]);
            try{
                ServeDrinkSummary serveDrinkSummary = vendingMachine.dispatch(buttonPressed, money);
                serveDrinkSummary.servedDrink.serveDrink();
                out.println(serveDrinkSummary.servedDrink.getName() + " " + serveDrinkSummary.change);
            }catch(OutOfStockException e){
                out.println(e.getMessage());
            }catch(InsufficientMoneyException e){
                out.println("Insufficient money");
            }
        }

        out.flush();
        out.close();
    }
}

class Coke implements Drink{
    private int price;
    private String name;
    private int quantityLeft;

    public Coke(int price, String name, int quantityLeft){
        this.price = price;
        this.name = name;
        this.quantityLeft = quantityLeft;
    }

    public int getPrice(){
        return this.price;
    }

    public String getName(){
```

```
        return this.name;
    }

    public int getQuantityLeft(){
        return this.quantityLeft;
    }

    public void serveDrink(){
        if(this.quantityLeft > 0)
            this.quantityLeft -= 1;
    }
}

class Fanta implements Drink{
    private int price;
    private String name;
    private int quantityLeft;

    public Fanta(int price, String name, int quantityLeft){
        this.price = price;
        this.name = name;
        this.quantityLeft = quantityLeft;
    }

    public int getPrice(){
        return this.price;
    }

    public String getName(){
        return this.name;
    }

    public int getQuantityLeft(){
        return this.quantityLeft;
    }

    public void serveDrink(){
        if(this.quantityLeft > 0)
            this.quantityLeft -= 1;
    }
}

class Sprite implements Drink{
    private int price;
    private String name;
    private int quantityLeft;
```

```java

    public Sprite(int price, String name, int quantityLeft){
        this.price = price;
        this.name = name;
        this.quantityLeft = quantityLeft;
    }

    public int getPrice(){
        return this.price;
    }

    public String getName(){
        return this.name;
    }

    public int getQuantityLeft(){
        return this.quantityLeft;
    }

    public void serveDrink(){
        if(this.quantityLeft > 0)
            this.quantityLeft -= 1;
    }
}
```

TESTCASE	DIFFICULTY	TYPE	STATUS	SCORE	TIME TAKEN	MEMORY USED
TestCase 0	Easy	Sample	Success	1	0.0524 sec	38 KB
TestCase 1	Easy	Hidden	Success	6	0.0598 sec	37.1 KB
TestCase 2	Easy	Hidden	Success	6	0.0686 sec	39.1 KB
TestCase 3	Easy	Hidden	Success	6	0.0867 sec	38.9 KB

TestCase 4	Medium	Hidden	Success	10	0.0653 sec	39.3 KB
TestCase 5	Medium	Hidden	Success	10	0.1108 sec	50.9 KB
TestCase 6	Medium	Hidden	Success	10	0.1302 sec	50.4 KB
TestCase 7	Hard	Hidden	Success	13	0.1717 sec	51.7 KB
TestCase 8	Hard	Hidden	Success	13	0.159 sec	50.3 KB

No comments.